

Exercises

Exercise: Graphical data exploration

1. Start RStudio on your computer. If you haven't already done so, create a new RStudio Project (select File → New Project on the main menu). Create the Project in a new directory by selecting 'New Directory' and then select 'New Project'. Give the Project a suitable name ('PU5058_stats' maybe) in the 'Directory name:' box and choose where you would like to create this Project directory by clicking on the 'Browse' button. Finally create the project by clicking on the 'Create Project' button. This will be your main RStudio Project file and directory which you will use throughout this course. See Section 1.6 of the Introduction to R book for more information about RStudio Projects and here for a short video.

Now create a new R script inside this Project by selecting File → New File → R Script from the main menu (or use the shortcut button). Before you start writing any code save this script by selecting File → Save from the main menu. Call this script 'graphical_data_exploration' or something similar. Click on the 'Files' tab in the bottom right RStudio pane to see whether your file has been saved in the correct location. Ok, at the top of almost every R script (there are very few exceptions to this!) you should include some metadata to help your collaborators (and the future you) know who wrote the script, when it was written and what the script does (amongst other things). Include this information at the top of your R script making sure that you place a # at the beginning of every line to let R know this is a comment. See Section 1.10 for a little more detail.

2. If you haven't already, download the data file '*loyn.xlsx*' from the **Data** link and save it to the **data** directory. Open this file in Microsoft Excel (or even better use an open source equivalent - LibreOffice is a good free alternative) and save it as a tab delimited file type. Name the file '*loyn.txt*' and also save it to the **data** directory.

3. These data are from a study originally conducted by Loyn (1987)¹ and subsequently re-analysed by Quinn and Keough (2002)² and Zuur et al (2009)³. Note, I have had to do some slight 'tweaking' of these data to improve usability for this course. The aim of the study was to relate bird density in 67 forest patches to a number of different environmental variables and management practices. A summary of the variables is: **ABUND**: Density of birds, continuous response variable; **AREA**: Size of forest patch, continuous explanatory variable; **DIST**: Distance to nearest patch, continuous explanatory; **LDIST**: Distance to nearest larger patch, continuous explanatory; **ALT**: Mean altitude of patch, continuous explanatory; **YR.ISOL**: Year of isolation of clearance, continuous explanatory; **GRAZE**: Index of livestock grazing intensity, 5 level categorical explanatory 1= low graze, 5 = high graze. Copy this information to your R script (make sure you comment it out with a # - can you remember the keyboard shortcut?) and clearly highlight which variable is the response variable and which variables are potential explanatory variables.

4. Import your tab delimited file from Q2 ('*loyn.txt*') into R using the `read.table()` function and assign it to an object called `loyn` (checkout Section 3.3.2 if you need a reminder). Use the `str()` function to display

the structure of the dataset and the `summary()` function to summarise the dataset. Copy and paste the output of `str()` and `Summary()` to your R code as a record. Don't forget to comment this code with a `#` at the beginning of each line (use the keyboard to comment code blocks shortcut?).

How many observations are in this dataset? How many variables does the dataframe contain?

Are there any missing values (coded as `NA`) in any variable?

How is the variable `GRAZE` coded? (as a number or a string?). If you think this will cause a problem (hint: it will!), create a new variable called **F`GRAZE` in the dataframe** with `GRAZE` recoded as a factor. See here to see how to convert/coerce a numeric variable into a factor (TL;DR: use the `as.factor()` or `factor()` function).

5. Use the function `table()` (or `xtabs()`) to determine how many observations were recorded for each **F`GRAZE` category (level)**. See section 3.5 of the Introduction to R book to remind yourself how to do this.

6. Using the `tapply()` function what is the mean bird abundance (`ABUND`) for each level of **F`GRAZE`**?

Can you also determine the variance for each **F`GRAZE` level**? Again see section 3.5 of the Introduction to R book to remind yourself how to do this.

7. Now onto some plotting action. Plot a Cleveland dotchart (Section 4.2.4) of each **numeric** variable separately to assess whether there are any outliers (unusually large or small values) in the response variable (`ABUND`) or any of the continuous explanatory variables (see Q3).

If you feel in the mood, output these plots to an external PDF file in an **output** directory within your RStudio project (don't forget to create the directory first if required). If you would like to include all of the plots on a single 'page' (technically a device) then you can split your page into two rows and three columns using `par(mfrow = c(2,3))` before you run your plot code for each plot. Make a note of which variables contain outliers.

8. If you do spot any unusual observations have a think about what you want to do with them (NOTE: do **not** just remove them without justification!). If you're unsure, please speak to an instructor to discuss your options during the practical session. Perhaps you should apply a data transformation to see if this reduces the magnitude of any outlier. The best thing to do here is to play around with different transformations (i.e. `log10`, `sqrt`) to see which transformation does what you want it to do. When applying a data transformation to a variable, it's best practice to create a new variable **in your dataframe** to contain your transformed variable rather than overwrite your original data.

After you have applied these data transformations make sure you re-plot your dotcharts with any transformed variable to double check whether the transformation is doing something sensible. Hint: a $\log_{10}$ transformation might help reduce the magnitude of the outliers for some of the variables.

9. Next, check if there is any potential collinearity between any of the **explanatory variables**. Remember, collinearity is *strong* relationships between your explanatory variables. Plot these variables using the `pairs()` function (Section 4.2.5).

You will need to extract your explanatory variables from the `loyn` dataframe (using `[]`) either before you use the `pairs()` function or whilst using it (don't forget to plot the transformed versions of any variables from Q8).

Optionally, include the correlation coefficient between variables in the upper panel of the pairs plot (see section 4.2.5 of the introduction to R book for details) to help you decide whether collinearity is an issue.

10. Now that we've checked for collinearity let's assess whether there are any clear relationships between the response variable (**ABUND**) and individual explanatory variables. Use appropriate plotting functions (**plot()**, **boxplot()**, **pairs()** etc) to visualise these relationships.

Don't forget, if you have applied a data transformation to any of your variables (Q8) you will need to plot these transformed variables instead of the original variables.

Also, don't forget, you can split your plotting device up to allow you to plot multiple graphs (Section 4.4) or again use a function like **pairs()** to create a multi-panel plot. Output these plots to the **output** directory as PDFs. Add some comments in your R code to summarise your findings.

11. One of the main aims of this study was to determine whether management practices such as grazing intensity (**GRAZE**) and size of the forest (**AREA**) affected the abundance of birds (**ABUND**). One hypothesis was that the size of the forest affected the number of birds, but this was dependent on the intensity of the grazing regime (in other words, there is an interaction between **AREA** and **GRAZE** - don't worry if you haven't heard of an interaction term, we will go through this later on in the course).

Use an appropriate plotting function to explore these data for such an interaction (perhaps a **coplot()** or **xypplot()** in Section 4.2.6 might be helpful?).

Again, don't forget, if you have applied a data transformation to your **AREA** variable you need to use the transformed variable in this plot **not** the original **AREA** variable. Likewise, as we've converted the **GRAZE** variable into a factor type variable we should use our new factor **FGRAZE** (or whatever you have called it).

Save this plot as a PDF to your **output** directory and add some comments to your R code to describe any patterns you observe.

¹ Loyn, R. (1987). Effects of patch area and habitat on bird abundances, species numbers and tree health in fragmented Victoria forests. *Nature conservation: the role of remnants of native vegetation*. 65-77.

² Quinn, G. P., and Michael J. Keough. 2002. *Experimental design and data analysis for biologists*. Cambridge, UK: Cambridge University Press.

³ Zuur, A.F., Ieno, E.N. and Elphick, C.S. (2010), A protocol for data exploration to avoid common statistical problems. *Methods in Ecology and Evolution*, 1: 3-14. doi:10.1111/j.2041-210X.2009.00001.x

End of Graphical data exploration Exercise